DataVisualisation.jl — how it works

Architecture, state, events (clicks and drags), buffers and connections · describes the code after commit `c247d1b` plus value-under-cursor, movie/PDF export and the run colour range (2026-10-19) · Julia 1.13, Bonito 5.2 · 642 tests

1. The big picture — layers
2. Start-up: from `explor_app` to the first figure
3. The state: `Dashboard`
4. Events: from a click or a drag to a render
5. The Debouncer — why nothing ever queues up
6. `render!` — the decision tree
7. Buffers: where the data lives and for how long
8. Clicks and drags on a time line
9. Navigating parameter space: snapping
10. Rules that keep it working
11. Where to change what · how to test it

The dashboard is a **Julia program that drives a web page**. All decisions are taken in Julia; the browser only shows widgets and figures and reports what the user does. Bonito keeps *observables* (values that notify listeners when they change) in sync between the two sides over a WebSocket; WGLMakie turns Makie figures into WebGL canvases on the page.

1. The big picture — layers

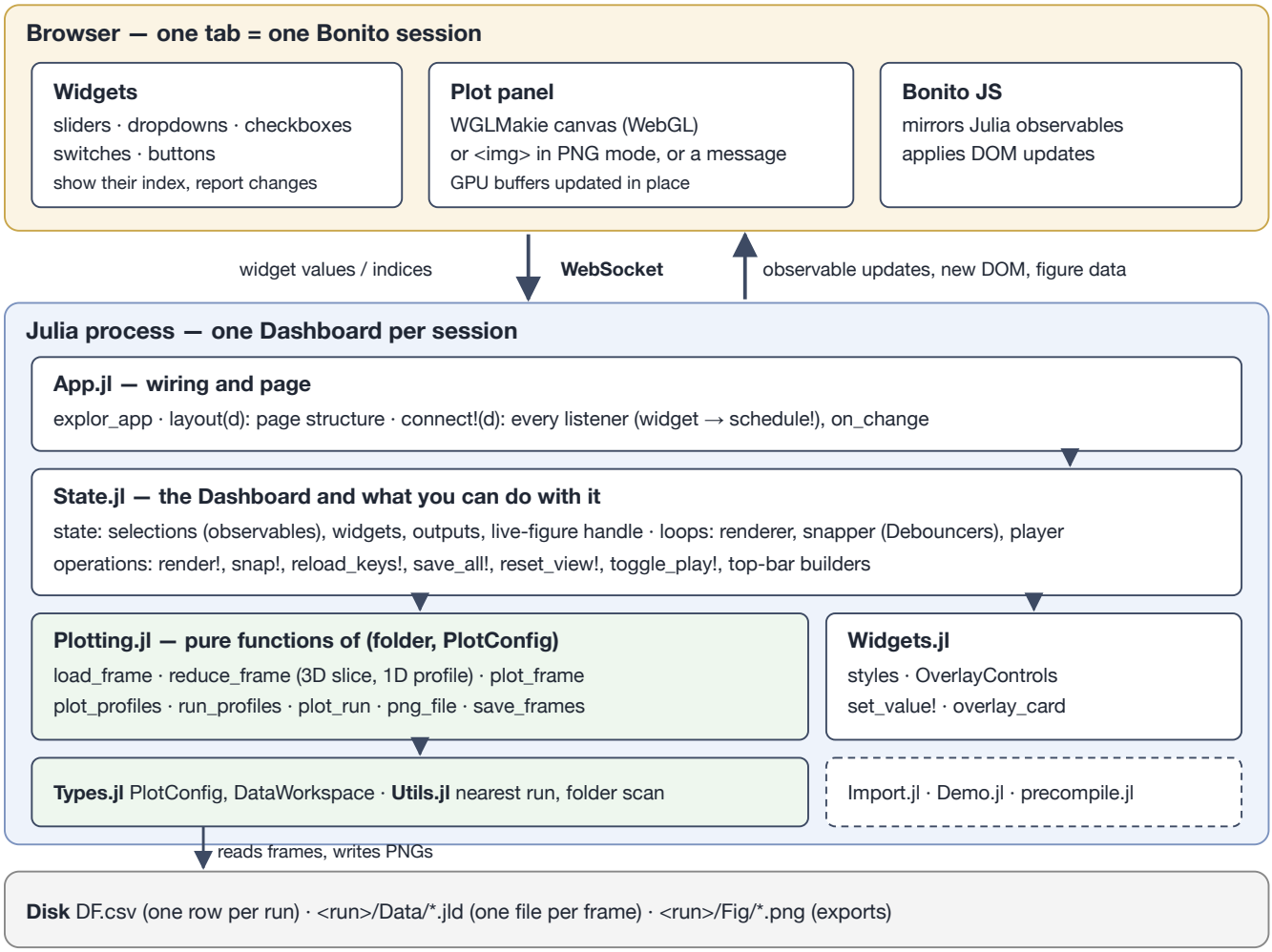


Figure 1 — Layers. Arrows point from caller to callee. Green boxes are pure (no widgets, no session): they are what the unit tests exercise directly.

The separation that matters: **App.jl only connects** (a listener never does work itself, it schedules it), **State.jl decides** (what to load, whether a figure can be updated in place), and **Plotting.jl computes** from a folder and a `PlotConfig` — a plain snapshot of every setting, built in one place (`plot_config(d)`). Rendering never reads a widget directly.

2. Start-up: from `explor_app` to the first figure

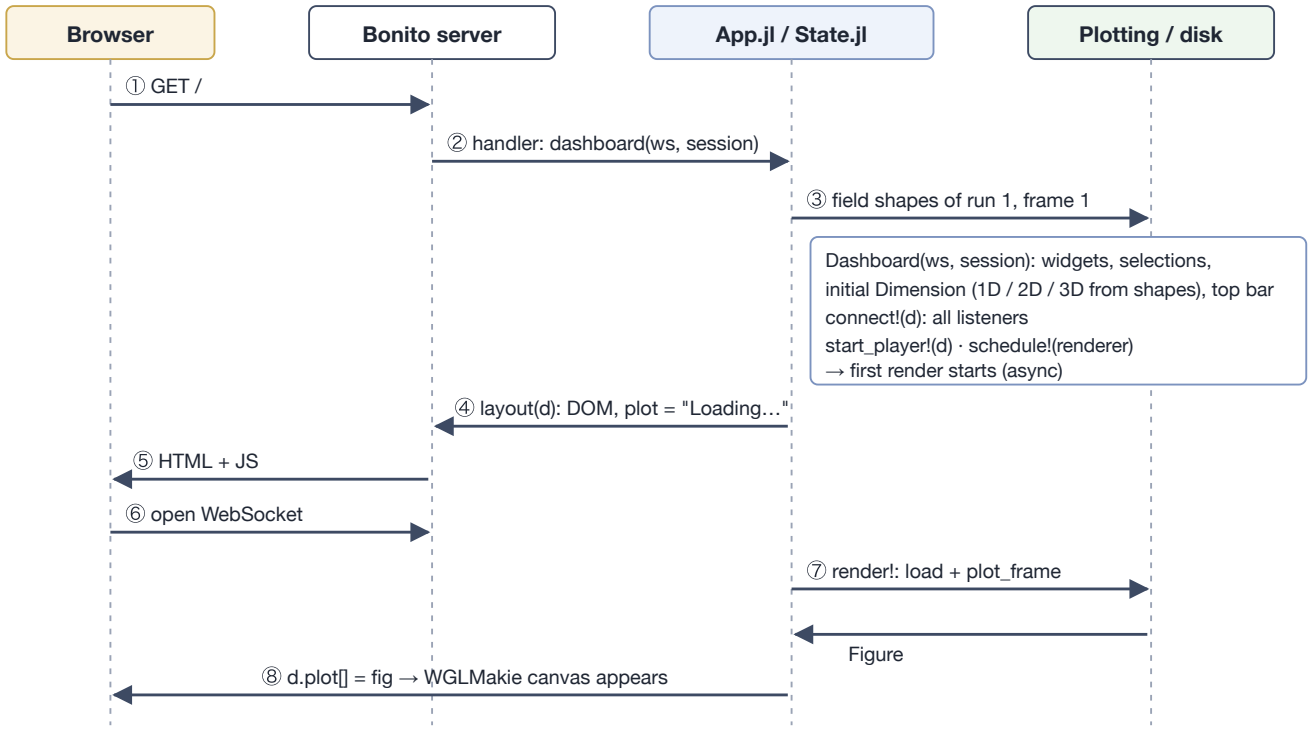


Figure 2 — Start-up of one session. ⑦ runs in a background task, so it can overlap ⑤–⑥; Bonito queues updates until the WebSocket is open.

- **Once per process**: `DataWorkspace(csv, ext)` reads `DF.csv`, finds the varying parameters, their value lists and ranges, the run folders, the number of frames and the field keys. It is shared by every session.
- **Once per browser tab**: Bonito calls the App handler, which builds a fresh `Dashboard` — two tabs never share widgets or figures.
- **Precompilation** (`precompile.jl`) runs steps ②–⑦ on a tiny generated ensemble when the package is installed, so the first real page takes ≈3 s instead of ≈10 s. It skips `connect!` and the player: their background tasks would block precompilation.

3. The state: `Dashboard`

Everything one session knows is one immutable struct of mutable things (observables, widgets, references). It is built by `Dashboard(ws, session)` and wired by `connect!`.

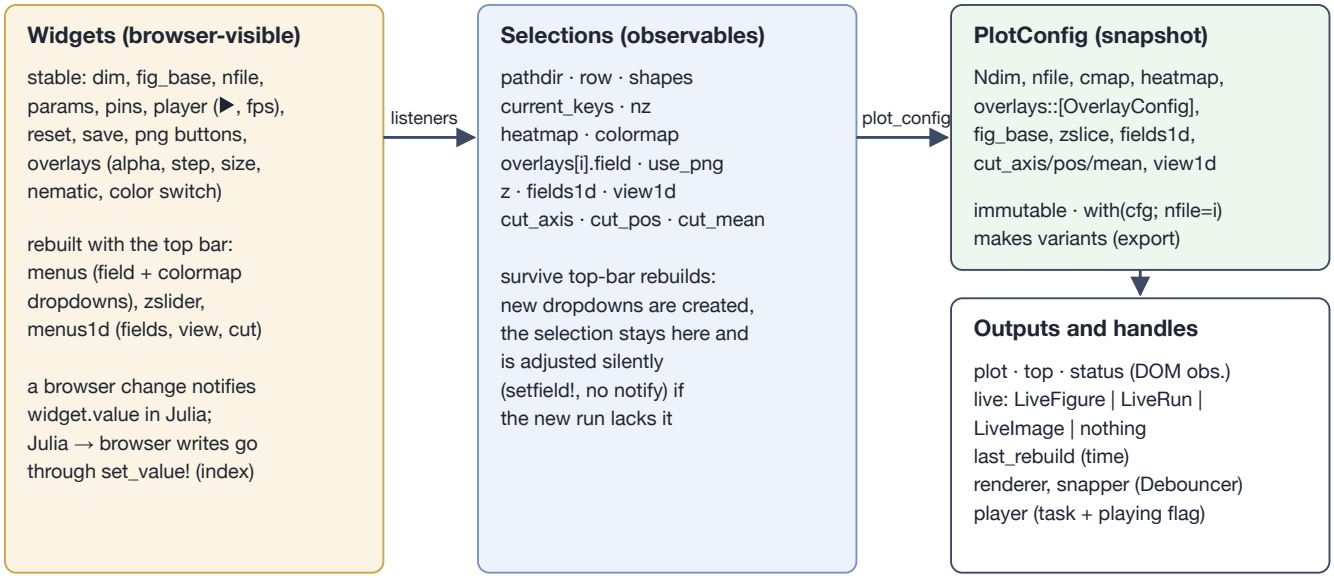


Figure 3 — What the Dashboard holds, and the one-way path widgets → selections → PlotConfig → render.

Group	Fields	Lifetime
Data source	`ws` (shared DataWorkspace), `session`	session
Current run	`pathdir`, `row` (DataFrame row), `shapes`, `current_keys`, `nz`	changes with the sidebar (snap)
What to plot	`heatmap`, `colormap`, `overlays`, `use_png`, `z`, `fields1d`, `cut_*`, `view1d`	survive top-bar rebuilds
Stable widgets	`dim`, `fig_base`, `nfile`, `params`, `pins`, `player`, buttons	session
Rebuilt widgets	`menus[]`, `zslider[]`, `menus1d[]` (Refs to the current ones)	until the next top-bar rebuild
Outputs	`plot` (Figure, <img> or message), `top`, `status`, `cursor` (value under the mouse)	session (contents replaced)
In-place handle	`live[]`: what is on screen and can be updated without a rebuild	until the next rebuild / `show!`
Loops	`renderer`, `snapper` (Debouncers), player task, `last_rebuild`	session (player stops when it closes)
Exports	`exporting`: one of Save all .png / Movie / PDF runs at a time	during an export

> **Index vs value.** Bonito's Slider and Dropdown display their *index*; `.value` is a Julia-side copy. A browser move updates both. A Julia write must go through `set_value!(widget, v)`, which sets the index — writing `.value` alone moves the label but not the thumb. Snapping and the player use `set_value!`.

4. Events: from a click or a drag to a render

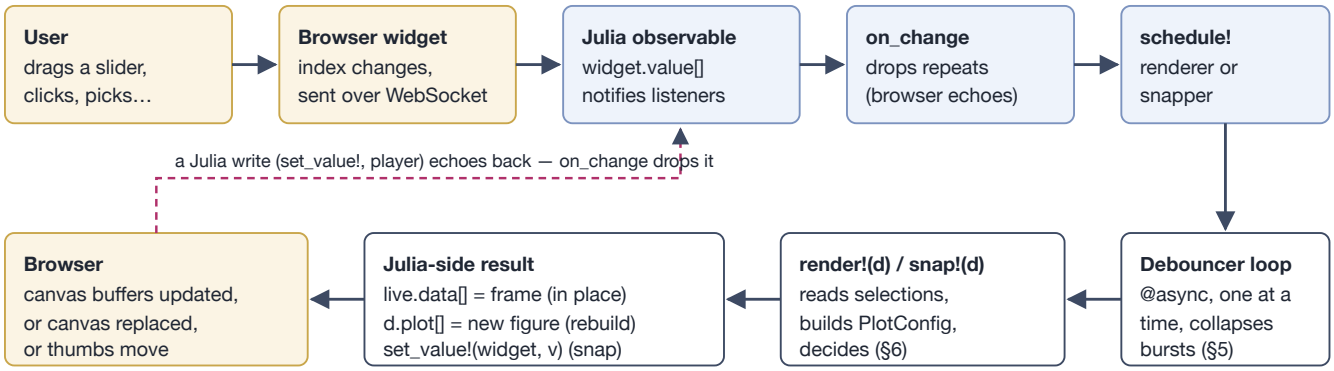


Figure 4 — The path of every user action. Buttons are simpler: a plain listener runs the operation in a background task (`@async(toggle_play!(d))` …).

Action	Listener → work	Figure
Time slider, ▶ playing	`nfile` → renderer	updated in place
Slice z (3D), cut position (1D)	`z`, `cut_pos` → renderer	in place
Arrow size	`overlays[i].arrowsize` → renderer	in place (`style`)
Run range (tick)	`fixed_range` → renderer → `request_range!` (background) → when ready, renderer again	rebuilt once when the range arrives; new run/field ranges in place
Field, colormap, overlay field / step / min brightness / nematic / color, Fig size, 1D fields / view / cut axis / mean	→ renderer	rebuilt, after the controls settle
Dimension	top bar rebuilt, then → renderer	rebuilt
Sidebar parameter or pin	→ snapper → maybe new `pathdir` → reload_keys! → renderer	in place if the new run has the same fields and sizes
Click ▶ / ▮▮	`toggle_play!`: flips `playing`, button label	—
Click ↺ Reset	`reset_view!`: reset both Debouncers, `show!("Reloading…")`, render	rebuilt
Click 💾 Save all .png	`save_all!`: `save_frames` with CairoMakie, progress in `status`	—
Click 🖼 Use PNG	`toggle_png!` → renderer	<img> or figure
Click 🎞 Movie / 📄 PDF	`save_movie!` (CairoMakie `record`, the figure updated in place frame by frame) / `save_pdf!`; errors go to `status`	— (files in `Fig/`)
Mouse move over the plot	listener set by `track_cursor!` on the figure: axis under the mouse → data coordinates → `cursor_text*` on the full-resolution frame (also on each new frame)	— (`cursor` line)

5. The Debouncer — why nothing ever queues up

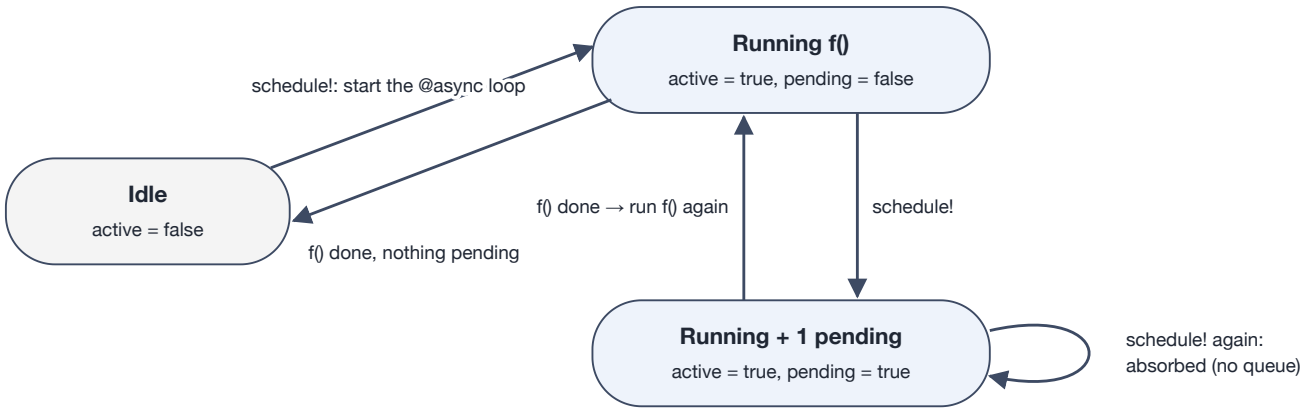


Figure 5 — Debouncer states. However many triggers arrive while `f` runs, exactly one more run follows, with the latest values. Two Debouncers exist: `renderer` (f = render!) and `snapper` (f = snap!).

Because `f` always reads the *current* selections, a burst of 50 slider events while one render runs gives one more render with the last value — never 50 queued renders, and the final position is never lost. `busy(renderer)` (active or pending) is also what the player waits on.

6. `render!` — the decision tree

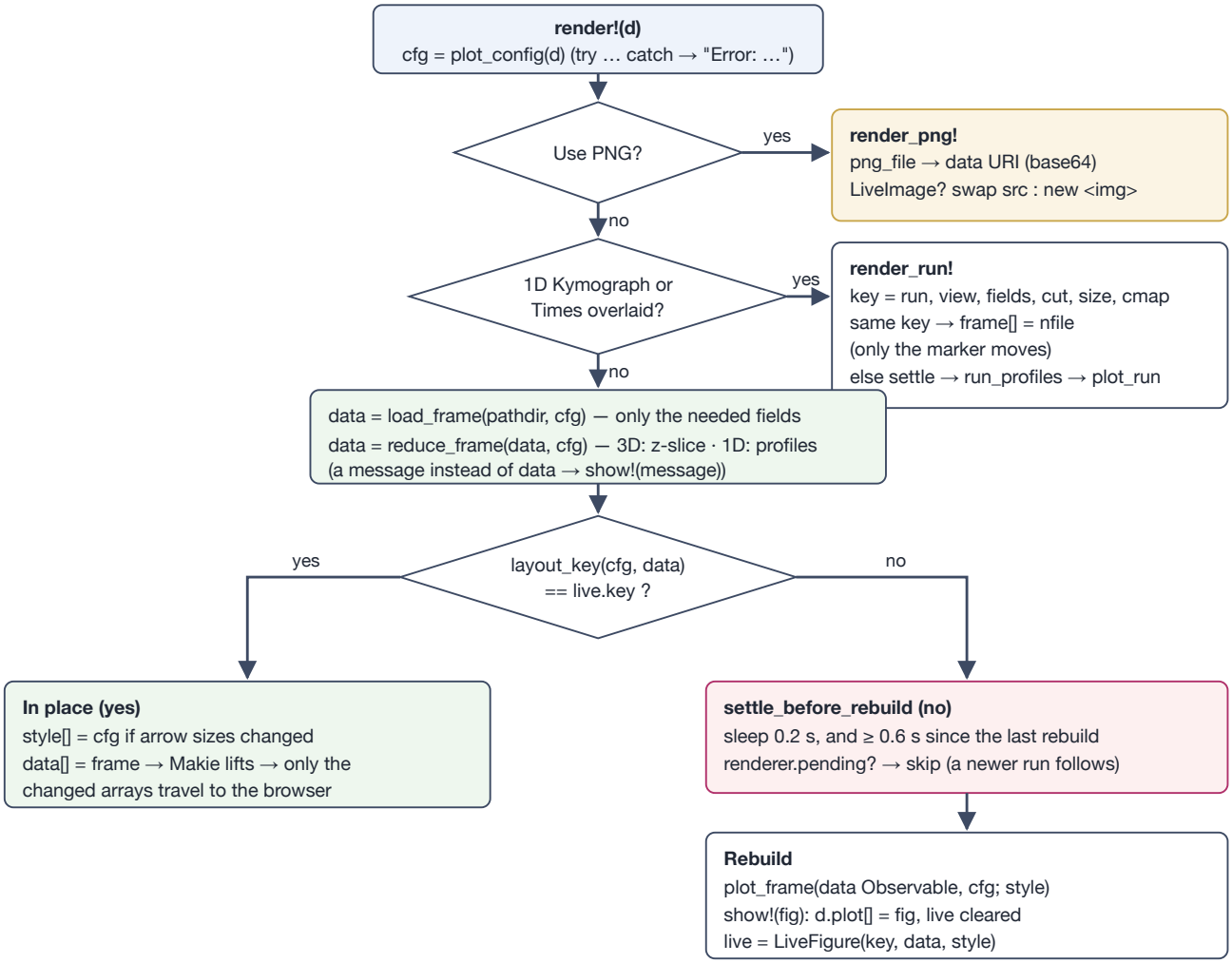


Figure 6 — Every render goes through this tree. `layout_key` = everything that shapes the figure: dimension, colormap, heatmap field, overlays (field, min brightness, step, nematic, colour — not size), Fig size, and the sizes of the arrays (1D: fields, cut axis/mean, Fig size, sizes).

7. Buffers: where the data lives and for how long

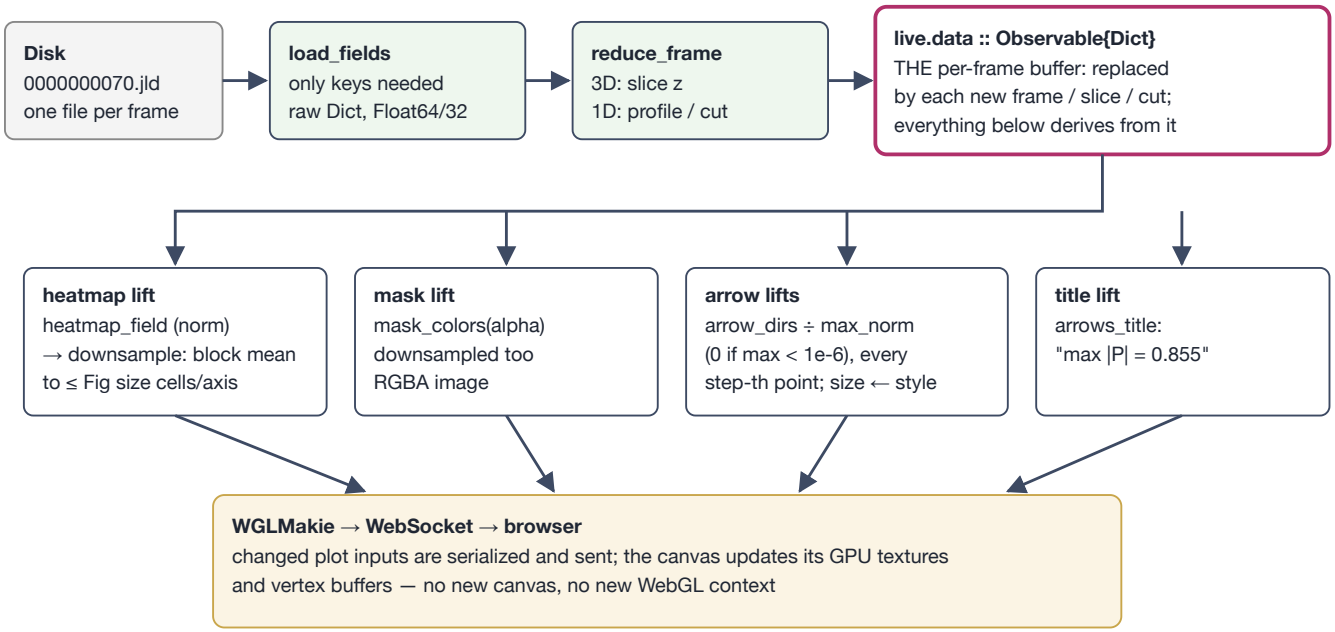


Figure 7 — The per-frame pipeline in data mode. Only the red-bordered buffer is replaced per frame; the lifts recompute and Makie ships only what changed. Exports (`save_frames`) use the same `plot_frame` at full resolution with CairoMakie.

Buffer / cache	Holds	Replaced when	Size, managed by
`DataWorkspace`	DF.csv, parameter tables and ranges, run folders, frame count, keys, colormap list	never (per process)	small · built once
`ws.run_ranges` (RangeCache)	colour range (min, max) per (run, field, dimension, frame count), and the computations in flight	never; a new frame count is a new key	a few numbers · filled in the background by `request_range!` → `run_range` (one field per file, another thread), shared by all tabs
`d.shapes`, `current_keys`, `nz`	field sizes of the current run's first frame	run changes (`reload_keys!`)	tiny
`LiveFigure.data`	current frame, reduced (slice / profiles), needed fields only	every frame, slice, cut	one frame · old Dict dropped (GC)
`LiveFigure.style`	the PlotConfig whose arrow sizes and run colour range are shown	arrow size or colour range changes	tiny
Makie plot inputs	downsampled heatmap, mask, arrow directions, title	derived from `data` / `style`	≤ Fig size² cells · Makie
`LiveRun` figure	whole run: field ⇒ (points × frames) matrix + times	run, view, fields, cut, Fig size or colormap changes	all frames of one run · loaded once
`LiveImage.src`	current PNG as a base64 data URI	every frame in PNG mode	one image
Browser canvas	one WebGL context per figure (GPU textures, vertex buffers)	rebuild only (old canvas removed)	browser · rebuilds settled (§8)
Debouncer flags, `last_rebuild`, player clock	scheduling state	continuously	a few numbers

> Nothing is cached *across* frames in data mode: each frame is read from disk (only the fields in use), reduced, and dropped when the next one arrives. Memory therefore stays at about one frame, whatever the run length — except in the 1D whole-run views, which hold the run's profiles by design.

8. Clicks and drags on a time line



Figure 8 — Four typical interactions. In-place work starts immediately; anything that replaces the WebGL canvas waits until the user is done (REBUILD_SETTLE = 0.2 s, MIN_REBUILD_GAP = 0.6 s).

> **Why the rebuild delay exists.** A rebuild replaces the canvas. Replacing it again before the browser has finished setting the previous one up makes WGLMakie throw (`setup_scene_init` on a removed canvas) and breaks the page until a refresh — this is what a fast drag of the arrow size did before it became an in-place update.

9. Navigating parameter space: snapping

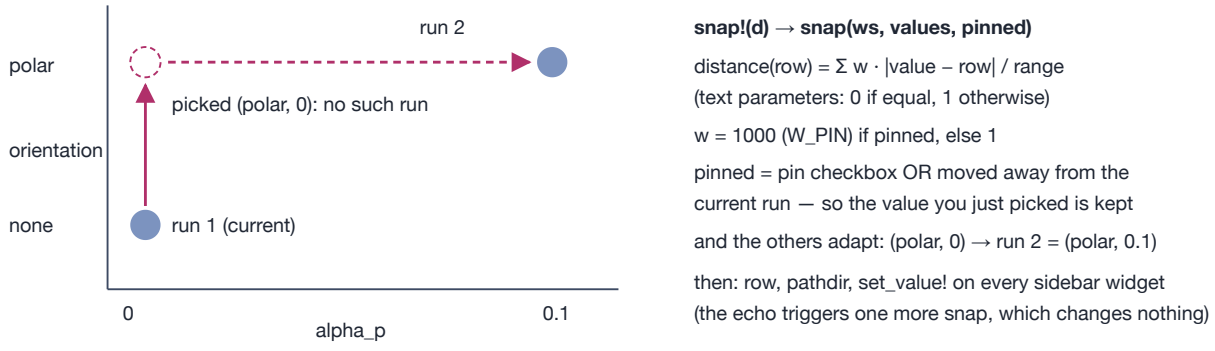


Figure 9 — Snapping in a parameter space with holes, as in PQ_FFT where `orientation=polar` only exists together with other parameter values.

10. Rules that keep it working

> **Listeners schedule, they don't work.** Writes to browser-synced observables from inside a listener go through `@async` or a Debouncer; writing while Bonito is still processing the current message causes "notify on null".

> **Julia → browser widget writes use** `set_value!` (index), never `.value` alone.

> **Listeners use** `on_change`: the browser echoes every value Julia sets; without the filter each animation frame would render twice.

> **Rebuilding the top bar adjusts selections silently** (`setfield!(obs, :val, …)`, no notify) and is always followed by one `schedule!(renderer)`.

> **A run change renders once**: `reload_keys!` (fields, slices, menus) first, then the render — never a render on the path change and another on the key change.

> **Anything that can change during playback must be an observable input of the figure** (data, style, frame marker), not part of `layout_key` — otherwise each change rebuilds the canvas. Anything in `layout_key` goes through `settle_before_rebuild`.

> **Makie attributes that follow data are passed as observables when the plot is created** (e.g. `Axis(…; title=obs)`); assigning an observable afterwards (`ax.title = obs`) is an error.

> **Nothing that spawns tasks goes into the precompile workload** (`connect!`, the player): precompilation would wait for them forever.

11. Where to change what · how to test it

To change…	Look at
what a frame looks like (heatmap, arrows, mask, title, 1D panels, kymograph)	`Plotting.jl`: `plot_frame`, `draw_overlay!`, `plot_profiles`, `plot_run`
how a field is reduced (3D slice, 1D cut, shape rules)	`Plotting.jl`: `depth`, `slice_field`, `profile`, `reduce_frame`
whether a change updates in place or rebuilds	`State.jl`: `layout_key`, `render_fields!`, `render_run!`, `LiveFigure.style`
a new setting	widget in `Dashboard/OverlayControls` → field in `PlotConfig` → `plot_config(d)` → `plot_triggers` → use it in Plotting; decide: in place or `layout_key`
which listener reacts to what	`App.jl`: `connect!`
page layout	`App.jl`: `layout`; top bars in `State.jl`: `top_dom`, `field_menus_dom!`, `slice_dom!`, `menus_1d_dom!`
run navigation	`Utils.jl`: `find_closest_row`, `snap`; `State.jl`: `snap!`
other data layouts / test data	`Import.jl` (`unstack_runs`), `Demo.jl` (`demo_dataset`)

Testing

- `julia --project=. -e 'using Pkg; Pkg.test()'` — 642 tests, ≈1 min, also run by GitHub Actions CI on every push. Pure functions are tested directly; the app is tested headlessly with `DV.dashboard(ws, Bonito.Session())`: setting a widget's value fires the same listeners a browser would.
- `julia dev/browser_check.jl <scenario> <DF.csv> <outdir> [fps]` — the real page in headless Chrome through the DevTools protocol: browser errors, frames per second, rebuilt canvases, screenshots. Scenarios: `load`, `polar`, `play`, `png-play`, `3d`, `1d`, `drag`, `hover`, `range`. It finds what the Julia tests cannot see (index/value desync, stuck spinners, WebGL errors).
- `julia demo.jl 1|2|3` — the generated ensemble (shapes, orders, anchorings) to try a change by eye.